An Industrial Training Report

On

FACIAL EMOTION DETECTOR

Submitted for partial fulfillment of award of

BACHELOR OF TECHNOLOGY

Degree in Information Technology

By

AMAN KUMAR

2300640130002

Under the Guidance of

Mr. Ajay Parashar

Assistant Professor,CSE

Department of Information Technology

Hindustan College of Science and Technology,

Farah, Mathura

DR. A.P.J. ABDUL KALAM UNIVERSITY, LUCKNOW, INDIA

December, 2025

HINDUSTAN COLLEGE OF SCIENCE & TECHNOLOGY

FARAH, MATHURA – 281122

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

Date: 16/12/2025

This is to certify that the **Industrial Training Report** entitled: **"Facial Emotion Detector"** submitted by **Aman Kumar** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech), Department of Information Technology, Hindustan College of Science and Technology, Farah, Mathura** is a bona fide work carried out under my supervision and guidance during the academic session **2025-26**.

Mr. Ajay Parashar	**Mrs. Deepti Mittal**
Assistant Professor, CSE	Head of Department, IT

ACKNOWLEDGEMENT

I express my sincere gratitude to **Hindustan College of Science and Technology, Farah, Mathura**, for providing me the opportunity to complete my industrial training and prepare this project report as a part of the B.Tech curriculum under the **Dr. A.P.J. Abdul Kalam Technical University, Lucknow**.

I owe my deepest appreciation to **Mr. Ajay Parashar, Assistant Professor, CSE**, for his continuous guidance, valuable suggestions, and encouragement throughout the course of this project. His support played a crucial role in shaping my understanding and improving the quality of this report.

I would also like to extend my heartfelt thanks to **Mrs. Deepti Mittal, Head of Department, IT**, for her constant motivation, academic support, and for providing a productive learning environment.

My sincere gratitude goes to **Oracle Cloud Infrastructure (OCI)** for offering me the internship opportunity, where I gained extensive hands-on experience in AI, Machine Learning, Deep Learning, Generative AI, and cloud-based deployments. The structured learning modules and practical exposure provided by OCI were invaluable in completing my project on **Facial Emotion Detection**.

Finally, I thank my faculty members, friends and family members who supported me—directly or indirectly—throughout this training period.

AMAN KUMAR

2300640130002

INTERNSHIP CERTIFICATE

ORACLE
University

ORACLE Certified Professional

Oracle Certified Professional

Certificate of Recognition

Aman Kumar

Oracle Cloud Infrastructure 2025 Certified Generative AI Professional

This certifies that the above named is recognized by Oracle Corporation as Oracle Certified.

October 30, 2025

Date

Damien Carey
Senior Vice President, Oracle University

This eCertificate is valid until October 30, 2027

102990123OCI25GAIOCP

PAGE INDEX

Topic			Page No.
I	CERTIFICATE		II
II	ACKNOWLEDGEMENT		III
III	INTERNSHIP CERTIFICATE		IV
1.	COMPANY PROFILE		1 – 3
	1.1	About Oracle cloud Infrastructure	1
	1.2	Company Work Culture and Environment	2
	1.3	Technologies used in the Organization	3
2.	TRAINING OVERVIEW		4 – 15
	2.1	Phases of the module	4
	2.2	Tools and Technologies used during training	12
	2.3	Summary	14
3.	PROJECT INTRODUCTION		16 - 25
	3.1	Introduction	16
	3.2	Problem Statement	17
	3.3	Feasibility Study	17
	3.4	Requirement Specifications	19
	3.5	System Overview	19
	3.6	Phases of Project	20
	3.7	Significance of the Project	25
	3.8	Summary	25
4.	SNAPSHOTS OF PROJECT MODULES		26-30

5.	CONCLUSION	31
6.	REFERENCES	32

CHAPTER – 1
COMPANY PROFILE

Oracle Cloud Infrastructure (OCI) is a leading cloud service platform developed by Oracle Corporation. It provides secure, scalable, high-performance computing resources for enterprises, developers, and researchers. OCI is used globally for hosting applications, managing databases, deploying AI/ML workloads, and performing large-scale data processing.

During my internship, OCI offered a structured and industry-standard environment for learning cloud computing, machine learning deployment, and advanced AI services. It follows modern DevOps and automation practices, enabling rapid development, testing, and deployment of cloud-native solutions. The platform emphasizes security, performance, and enterprise-grade reliability, making it suitable for real-world production systems.

OCI's work culture encourages students and professionals to innovate, explore cloud technologies, and gain hands-on experience with cutting-edge tools like Oracle Vision AI, Object Storage, Functions, and Compute Instances.

1.1 ABOUT ORACLE CLOUD INFRASTRUCTURE

Oracle Cloud Infrastructure is Oracle's second-generation cloud platform designed to support highly available AI and enterprise applications. It provides a broad set of services such as:

- Compute services for running virtual machines and GPU-based workloads
- Autonomous Database for automated, self-tuning data storage

- AI and Machine Learning services including Vision AI and Speech AI
- Networking, Storage, and Security services
- Serverless computing (Oracle Functions)
- Developer tools such as DevOps pipelines and API gateways

OCI emphasizes high performance, fault tolerance, and secure architecture. It is used by global industries like banking, healthcare, telecommunications, and government sectors due to its reliability and compliance standards.

The organization encourages learning through structured training modules, hands-on labs, and real-time project tasks, helping students understand industry cloud standards.

1.2 COMPANY WORK CULTURE AND ENVIRONMENT

Oracle follows a collaborative and research-driven work culture. Teams operate in Agile methodology, where tasks are divided into sprints, and continuous integration/continuous deployment (CI/CD) is practiced.

Interns are provided access to cloud sandboxes, documentation, and mentorship that help them apply theoretical knowledge to real-world cases.

Major highlights of the work culture include:

- Structured workflows and documentation
- Exposure to enterprise cloud architecture
- Regular team discussions, updates, and knowledge-sharing sessions
- Focus on performance optimization, data security, and solution scalability

This environment helps learners gain practical and professional skills.

1.3 TECHNOLOGIES USED IN THE ORGANIZATION

Oracle Cloud Infrastructure uses modern and advanced technologies to deploy and manage large-scale applications:

- **Machine Learning & Vision AI**
  Used for analysis, image recognition, object detection, and emotion classification.
- **Oracle Compute Instances (VM and GPU)**
  Run ML models and backend services.
- **Object Storage**
  Stores training datasets, model outputs, logs, and media files.
- **Oracle Functions (Serverless)**
  Executes event-driven ML inference tasks.
- **Databases (Autonomous DB, NoSQL DB)**
  Stores user data, prediction logs, and application metadata.
- **Developer Tools**
  DevOps pipelines, API gateways, monitoring dashboards.

Overall, Oracle ensures a modern, secure, scalable environment aligned with real industry practices.

CHAPTER – 2
TRAINING OVERVIEW

During my internship at Oracle Cloud Infrastructure (OCI), I underwent a structured and industry-oriented training program focused on Artificial Intelligence (AI), Machine Learning (ML), Deep Learning (DL), and Generative AI (GenAI). The training was divided into eight core modules, each designed to build my understanding step by step—from AI basics to advanced LLM workflows and Generative AI agent development.

This structured learning pathway allowed me to gradually progress through conceptual foundations, practical labs, cloud-based implementation, and finally integrating Oracle's GenAI services into real project development.

Below is the complete training overview rewritten according to your modules:

2.1 PHASES OF THE MODULE

2.1.1 – AI Foundations

The internship began with Artificial Intelligence (AI) fundamentals, focusing on the evolution, scope, and real-world relevance of AI in modern computing systems. This module introduced how AI systems differ fundamentally from traditional rule-based programming by enabling machines to learn, reason, and adapt from data. Core concepts such as intelligent agents, search strategies, problem-solving methods, and ethical implications of AI were discussed in depth.

Special emphasis was given to understanding how AI is applied across multiple domains, including healthcare diagnostics, financial fraud detection, smart

agriculture, surveillance systems, and recommendation engines. The concept of environment modeling and agent-based decision-making helped establish how intelligent systems interact with real-world scenarios. Ethical concerns such as bias, fairness, transparency, and responsible AI usage were also highlighted.

Key Learnings:

- Definition and scope of Artificial Intelligence
- Difference between traditional programming and AI-based systems
- Types of AI: ANI, AGI, and ASI
- Real-world AI applications
- Intelligent agent design and environment modeling
- Learning paradigms: supervised, unsupervised, and reinforcement learning
- Understanding datasets, features, and labels

Insight:

This phase aligned my theoretical understanding with industry practices and created a strong conceptual base, which proved essential for progressing toward Machine Learning and Deep Learning concepts used in my project.

2.1.2 – Machine Learning foundations

This module focused on the core principles of Machine Learning (ML), emphasizing how systems learn patterns from data and make predictions without explicit programming. The complete ML lifecycle was explored, starting from data collection and preprocessing to model training, evaluation, and deployment.

Various learning paradigms such as classification, regression, clustering, and recommendation systems were studied. Practical understanding of algorithms like Linear Regression, Logistic Regression, Decision Trees, Support Vector Machines, and K-means clustering was developed. Considerable attention was

given to handling real-world data challenges such as noise, missing values, and feature scaling.

Model evaluation techniques and performance metrics were studied to ensure reliable and unbiased predictions. Concepts like overfitting, underfitting, and the bias–variance tradeoff were analyzed to understand how models generalize to unseen data.

Key Learnings:

- Supervised and unsupervised learning techniques
- Classification, regression, clustering algorithms
- Model evaluation metrics (Accuracy, Precision, Recall, F1-score, RMSE)
- Feature scaling, normalization, and encoding
- Bias–variance tradeoff and generalization

Insight:

This module provided a practical understanding of how models learn from data, forming the foundation for training the facial emotion classification model used in the project.

2.1.3 – Deep Learning Foundations

Deep Learning concepts were introduced to handle complex data patterns, particularly for computer vision tasks such as facial emotion recognition. This module explained how artificial neural networks with multiple hidden layers can automatically extract high-level features from data.

The working of neurons, activation functions, forward propagation, backpropagation, and gradient descent optimization was studied in detail. Special focus was given to Convolutional Neural Networks (CNNs), which are widely

used in image-based tasks, and Recurrent Neural Networks (RNNs) for sequential data.

Hyperparameter tuning strategies were explored to improve model performance and reduce training errors.

Key Learnings:

- Artificial Neural Networks architecture
- Activation functions (ReLU, Sigmoid, Softmax)
- CNNs for image processing
- RNNs and LSTMs for sequence learning
- Training and optimization techniques
- Hyperparameter tuning

Insight:

This module directly supported my project, as deep learning concepts explain how facial features can be learned and classified effectively.

2.1.4 – Generative AI & LLM Foundations

This module introduced Generative Artificial Intelligence, which represents a major advancement in AI systems by enabling machines to generate new and meaningful content rather than only performing prediction or classification. The training focused on understanding how generative models learn the underlying data distribution and use it to create outputs such as text, images, embeddings, and synthetic data.

The fundamentals of Large Language Models (LLMs) were covered, explaining how these models are trained on massive datasets using transformer-based architectures. Concepts such as embeddings, contextual understanding, and

probabilistic text generation were discussed to explain how LLMs produce coherent and context-aware responses. Various generative architectures, including Generative Adversarial Networks (GANs), autoencoders, and diffusion models, were introduced to demonstrate different approaches to content generation.

Special attention was given to prompt engineering, which plays a crucial role in controlling and optimizing LLM outputs. Ethical considerations such as hallucination, data bias, misinformation, and responsible AI usage were also emphasized to ensure safe deployment of generative systems.

Key Learnings:

- Fundamentals of Generative AI and content creation
- GANs, autoencoders, and diffusion models
- Embeddings and vector representations
- Prompt engineering techniques
- Capabilities and limitations of LLMs
- Ethical and safety considerations in Generative AI

Insight:

This module helped me understand how Generative AI enhances traditional AI systems by enabling intelligent content generation, dataset augmentation, and advanced human–computer interaction, which are highly relevant in modern AI applications.

2.1.5 – AI Portfolio Development

This phase focused on transforming technical knowledge into professionally structured AI projects suitable for academic evaluation and industry presentation. The training emphasized how real-world AI solutions are designed, documented, evaluated, and maintained over time.

Students were guided through the complete end-to-end AI project lifecycle, starting from problem identification and dataset selection to model training, evaluation, optimization, and final deployment. Importance was given to maintaining clean project structure, readable code, reproducibility, and proper documentation. The role of evaluation metrics and validation techniques in proving model reliability was discussed in detail.

Additionally, this module highlighted how AI projects should be presented through reports, Git repositories, and portfolios to communicate technical competence effectively to recruiters and evaluators.

Key Learnings:

- End-to-end AI project development workflow
- Dataset selection and justification
- Model evaluation, validation, and reporting
- Reusable and modular ML pipelines
- Documentation and version control practices
- Professional AI portfolio building

Insight:

This phase directly influenced the organization and presentation of my Facial Emotion Detection System, ensuring it followed academic standards and professional software engineering practices.

2.1.6 – Generative AI Services on Oracle Cloud

This module provided hands-on exposure to Oracle Cloud Infrastructure (OCI) and its AI-powered services. The training focused on how cloud platforms simplify AI deployment, scalability, and real-time inference. Oracle's Generative AI services were explored to understand how pre-trained models can be integrated into applications using APIs.

Special emphasis was placed on Oracle Vision AI, which is capable of performing tasks such as face detection, object recognition, and emotion classification. The use of Object Storage for managing datasets, model files, and outputs was demonstrated. Students also learned how cloud-based workflows improve reliability, performance, and accessibility of AI systems.

Integration of AI services with cloud functions enabled automated inference pipelines and real-time processing.

Key Learnings:

- Oracle Generative AI APIs
- Vision AI services for facial analysis
- Cloud-based dataset and model management
- Object Storage usage
- Integration of AI outputs with cloud functions

Insight:

This module directly supported the cloud-oriented understanding of AI systems and strengthened my ability to deploy and manage emotion detection workflows using enterprise-grade infrastructure.

2.1.7 – Fundamentals of Large Language Models (LLMs)

This module focused on the internal working of Large Language Models, providing a deeper understanding of how modern AI systems process and

generate language. The Transformer architecture was explained in detail, including attention mechanisms, multi-head attention, and positional encoding.

Students learned how raw text is converted into tokens and embeddings and how contextual meaning is preserved across long sequences. The differences between fine-tuning, prompt-tuning, and instruction-based learning were discussed to highlight various adaptation strategies used in industry.

Use cases such as text classification, summarization, question-answering, and conversational AI were analyzed along with methods to benchmark LLM performance.

Key Learnings:

- Transformer architecture and attention mechanism
- Tokenization and embeddings
- Fine-tuning vs prompt-tuning
- LLM applications and benchmarking
- Limitations and optimization strategies

Insight:

This module strengthened my understanding of modern AI workflows and improved my ability to design intelligent systems that leverage language understanding and contextual reasoning.

2.1.8 – Generative AI Agent Services

The final module focused on building AI-driven agents capable of autonomous reasoning, decision-making, and task execution. These agents combine LLMs with external tools, APIs, and memory systems to perform complex workflows.

Students learned how AI agents manage state, plan actions, and interact dynamically with users. Workflow automation using Generative AI was demonstrated, highlighting how agents can respond to user inputs, retrieve data, and adapt behavior based on context. Concepts such as tool invocation, agent orcestration, and feedback loops were explored.

Emotion-aware and context-sensitive agents were discussed, linking directly to applications like emotion recognition systems and intelligent assistants.

Key Learnings:

- AI agent architecture and design
- Integration of LLMs with tools and APIs
- State management and reasoning
- Workflow automation using Generative AI
- Emotion-aware and adaptive agents

Insight:

This concluding phase integrated all concepts learned throughout the training—AI, ML, DL, LLMs, and cloud services—into a unified framework, preparing me to build advanced, intelligent, and emotion-aware applications aligned with my project.

2.2 TOOLS AND TECHNOLOGIES USED DURING TRAINING AND PROJECT DEVELOPMENT

The successful implementation of the Facial Emotion Recognition System required the use of several software tools, programming libraries, and data platforms. These tools supported different phases of the project, including data collection, preprocessing, facial feature extraction, model training, evaluation,

and real-time emotion detection. The following tools and technologies were extensively used throughout the training and development process:

2.2.1 Python

Python served as the primary programming language for the entire project. It was chosen due to its simplicity, readability, and extensive support for machine learning and computer vision applications.

2.2.2 OpenCV (Open Source Computer Vision Library)

OpenCV was used for real-time image and video processing tasks. It enabled webcam access, frame capture, face detection, and visualization of emotion predictions. OpenCV plays a crucial role in converting raw video input into processable frames and displaying results in real time.

2.2.3 NumPy

NumPy was used for numerical computations and matrix operations throughout the project. Facial landmark coordinates extracted from images were stored and processed using NumPy arrays.

2.2.4 Pandas

Pandas was used for data handling, preprocessing, and dataset management. The facial landmark data extracted from images was stored in structured CSV files and managed using Pandas DataFrames.

2.2.5 Kaggle

Kaggle served as the primary source for the facial emotion dataset used in the project. Kaggle provides high-quality, publicly available datasets suitable for academic research and machine learning experimentation. The dataset obtained from Kaggle contained labeled facial images representing different emotions, which were used for landmark extraction and model training.

2.2.6 MediaPipe

MediaPipe was utilized for efficient facial landmark detection and face mesh extraction. It provides a fast and accurate framework for real-time face analysis, making it suitable for emotion recognition tasks. MediaPipe's face mesh model detects multiple facial landmarks with high precision, enabling robust feature extraction even under varying lighting and facial orientations. Its lightweight design allows real-time execution on standard hardware.

Supporting Libraries and Tools

- In addition to the core tools, several supporting technologies were used:
- scikit-learn – For implementing the MLP Classifier, model training, evaluation, and serialization
- Jupyter Notebook / Google Colab – For experimentation, visualization, and iterative model development
- Pickle (.pkl files) – For saving trained models and label encoders
- Matplotlib / Seaborn (optional) – For visualizing data distributions and model performance

2.3 SUMMARY OF TRAINING EXPERIENCE

Across these eight structured modules, I gained hands-on experience in:

- AI fundamentals & problem-solving
- Machine learning workflow creation
- Deep learning for image-based models
- Generative AI & LLM architectures
- Building and deploying cloud-based AI solutions

- Using Oracle's GenAI services for real-world applications
- Designing intelligent agents
- Managing datasets and deploying ML systems at scale

This training empowered me to independently design and develop the Facial Emotion Detection System using OCI, applying both theoretical knowledge and industry best practices.

CHAPTER – 3
PROJECT INTRODUCTION

Facial Emotion Detection System using Machine Learning and Python:

3.1 INTRODUCTION

Facial expressions are one of the most natural and powerful ways through which humans communicate emotions. Understanding emotions from facial features plays a significant role in numerous applications, including mental health analysis, human–computer interaction, surveillance systems, online learning environments, customer behavior monitoring, and entertainment systems. With rapid advancements in machine learning, computer vision, and artificial intelligence, automated facial emotion detection has become a highly researched topic.

This project, **Facial Emotion Detection System using Python**, focuses on identifying human emotions in real time by analyzing facial landmarks. The aim is to build a lightweight, efficient, and reliable emotion classification model that can process live video input from a webcam and classify basic emotional states such as happiness, anger, sadness, fear, disgust, surprise, and neutrality. The project is built using **Python**, with key libraries such as **scikit-learn**, **OpenCV**, **dlib**, **NumPy**, and **Pandas**.

The entire system was developed in three well-defined phases: **landmark extraction**, **model training**, and **real-time emotion detection**. Each phase required a different set of technologies and skills, ranging from data preprocessing and feature engineering to model building, serialization, and real-time computer vision. The project demonstrates how classical machine learning

algorithms can be combined with modern computer vision techniques to create a fast and practical facial emotion detection pipeline.

3.2 PROBLEM STATEMENT

Emotions form an essential aspect of human communication, but they are often difficult to quantify or observe through digital systems. Traditional methods require manual labeling or interpretation by humans, which is time-consuming and subjective. There is a need for an automated, real-time solution capable of identifying emotions accurately based on facial cues.

Many advanced deep learning models can perform facial emotion recognition; however, they often require high computational power and large datasets. The objective of this project was to create an **efficient, lightweight model** using **facial landmarks instead of raw images**—reducing computational cost while preserving useful information about facial geometry.

By extracting landmarks and using them as features, the system remains fast, interpretable, and resource-efficient—an ideal combination for real-time applications on standard hardware.

3.3 FEASIBILITY STUDY

A feasibility study was conducted to evaluate the practicality and effectiveness of implementing the proposed **Facial Emotion Recognition System**. The study examines the system from technical, economic, and operational perspectives to ensure that the project is viable for real-world and academic implementation.

3.3.1 Technical Feasibility

The project is technically feasible as it is developed using **Python** along with well-established and widely supported libraries such as **OpenCV, dlib, scikit-learn, NumPy, and Pandas**. These technologies provide all the necessary

functionalities for image processing, facial landmark extraction, machine learning model training, and real-time emotion detection.

OpenCV enables real-time video capture and face detection, while dlib provides an accurate facial landmark predictor essential for feature extraction. Scikit-learn is used to implement the **MLP Classifier**, which efficiently classifies emotions based on facial landmarks. The system can be executed on standard hardware without requiring GPU acceleration, making it suitable for academic environments. Therefore, from a technical standpoint, the project can be implemented reliably and maintained effectively.

3.3.2 Economic Feasibility

The project is economically feasible as it relies entirely on **free and open-source tools**. Python and all the supporting libraries used in the project are freely available and do not involve any licensing costs. The facial emotion dataset used for training was obtained from open platforms such as **Kaggle**, which are accessible at no cost for academic purposes.

Additionally, the system does not require expensive hardware, cloud services, or paid APIs. It runs efficiently on a standard personal computer with a webcam, minimizing overall development and operational costs. Hence, the project is cost-effective and suitable for student-level implementation.

3.3.3 Operational Feasibility

The Facial Emotion Recognition System is operationally feasible due to its **simple and user-friendly interface**. The system automatically captures facial input through a webcam and displays the detected emotion in real time, requiring minimal user interaction.

Users do not need any technical expertise to operate the system, as the process is fully automated once the application is launched. The system requires minimal

training and can be easily deployed in real-world scenarios such as emotion analysis, human–computer interaction, and educational demonstrations. Therefore, the system is practical, easy to use, and operationally efficient.

3.4 REQUIREMENT SPECIFICATIONS

Requirement specification defines the **functional and non-functional requirements** of the Facial Emotion Recognition System.

3.4.1 Functional Requirements

- Capture live video input using a webcam
- Detect human faces from the video stream
- Extract facial landmarks from detected faces
- Classify facial expressions into predefined emotion categories
- Display detected emotion in real time
- Load and utilize trained machine learning model for prediction

3.4.2 Non-Functional Requirements

- Fast response time for real-time emotion detection
- High accuracy and reliability of emotion classification
- Secure handling of dataset and model files
- Scalability for future enhancements such as deep learning integration
- Compatibility with standard operating systems and webcams

3.5 SYSTEM OVERVIEW

The proposed system follows a structured workflow:

1. **Dataset Collection & Preparation**

   A facial emotion dataset was obtained from Kaggle. The dataset

contained images belonging to different emotion categories such as happy, sad, angry, surprised, etc. These images formed the foundation for the feature extraction and model training phases.

2. **FacialLandmark Extraction**
   Instead of training directly on images, which is computationally expensive, the system extracts a set of **68 facial landmarks** for each image. These landmarks represent key points on the eyes, eyebrows, nose, mouth, and jawline. The coordinates of these landmarks capture detailed geometric relationships that correspond strongly with specific emotions.
3. **DatasetConversion to Landmark Features:**
   All extracted landmarks were saved in a structured .csv file—each row representing one image, and each column representing a coordinate (x or y) of a landmark point. This dataset becomes the input for the machine learning model.
4. **Model Training using Scikit-learn**
   Classical ML algorithms were explored, and the model was trained using scikit-learn. The training process included label encoding, data splitting, feature standardization, and algorithm selection.
5. **Model Serialization**
   Once trained, the model was saved as a .pkl file, and label encoders were saved so the system could decode predictions during real-time execution.
6. **Real-Time Emotion Recognition**
   The model was integrated with **OpenCV**, enabling it to capture frames from the webcam, detect faces, extract landmarks, and classify emotions instantly.

3.6 PHASES OF PROJECT

3.6.1 Phase 1 – Facial Landmark Extraction

This was the foundational step of the entire project. Instead of training the model on raw images—which require deep neural networks and GPU acceleration—the project utilizes **facial landmarks**, a more computationally efficient representation of visual data.

Steps Involved in Landmark Extraction:

1. **Loading Dataset From Kaggle**

   The dataset contained thousands of facial images categorized by emotion. Each category was placed in its own folder.

2. **Using dlib for Landmark Detection**

   The dlib library provides a powerful 68-point facial landmark predictor. For each image:

   - The face is detected using a frontal face detector.
   - The landmark predictor identifies 68 distinct points on the face.
   - These points correspond to jaw, eyebrows, eyes, nose, and lips.

3. **Extracting Landmark Coordinates**

   For each of the 68 landmarks, both **x** and **y** coordinates were stored, resulting in **136 numerical features per image**. This geometric representation is highly effective for emotion classification.

4. **Creating landmarks.csv**

   The extracted features were saved into a .csv file along with a label column representing the emotion category.

Why Landmark Extraction?

- Faster than deep learning-based raw image training
- Very lightweight and efficient
- Emotion-related facial deformations are strongly captured through geometry
- Suitable for real-time applications using simple ML models

This stage transformed raw data into a highly structured feature set that could be easily used for machine learning.

3.6.2 Phase 2 – Model Training Using Scikit-Learn (MLP Classifier)

After preparing the landmark-based dataset, the next major stage of the project involved training a machine learning model capable of accurately classifying human emotions. For this purpose, the MLP Classifier (Multi-Layer Perceptron) from scikit-learn was chosen. Unlike simple traditional classifiers, an MLP is a form of feed-forward neural network that can learn highly complex non-linear patterns, making it ideal for identifying subtle changes in facial geometry.

1. Loading the Landmark Dataset

Using Pandas, the landmarks.csv file containing 136 landmark features and one label column was loaded. Each row represented the facial landmarks of a single image and the corresponding emotion.

2. Label Encoding of Emotions

Because the output labels were categorical (e.g., *happy*, *sad*, *angry*, *neutral*), scikit-learn'sLabelEncoder was used to convert them into numeric form.

3. Splitting the Dataset

The dataset was divided into:

- 80% for training the MLP model
- 20% for testing and validation

This ensured unbiased evaluation of the model.

4. Choosing MLP Classifier

The MLP Classifier was selected because:

- It can learn non-linear patterns in landmark coordinates
- It supports hidden layers, making it more expressive
- It performs well on structured datasets
- It does not require GPU training
- It is fast and suitable for real-time applications

An example architecture used in MLP:

- Input layer: 136 neurons (each landmark x,y coordinate)

- Hidden layer(s): Example 100–150 neurons
- Output layer: number of emotion classes (e.g., 5 or 7)

Activation functions like ReLU and tanh, along with solvers like adam, were explored depending on performance.

5. Training the Model

The MLP classifier was fitted on the training data:

- The model learned how landmark patterns differ across emotions
- The training process optimized internal weights using backpropagation
- Loss was minimized until accuracy stabilized

MLP required careful tuning:

- Epochs
- Hidden layer sizes
- Activation functions
- Learning rate

Through experimentation, the classifier achieved stable accuracy on the validation set.

6. Model Evaluation

Once training was complete, performance metrics such as accuracy, classification reports, confusion matrices, and error trends were analyzed. The model demonstrated strong capability in recognizing emotion-specific geometric facial changes, validating the effectiveness of MLP for this task.

7. Saving the Model & Label Encoder

To enable real-time recognition without retraining:

- The trained MLP model was saved as a .pkl file
- The label encoder was saved as a separate .pkl file

These files could be loaded instantly during real-time detection.

Why MLP Classifier Was a Good Choice

- Works well on structured numerical data
- Captures complex emotional patterns from landmark geometry

- Lightweight and fast during inference
- More flexible than linear models
- Requires no GPU and runs smoothly in real time

Using MLP ensured the system remained efficient, memory-friendly, and capable of high-accuracy predictions.

3.6.3 Phase 3 – Real-Time Emotion Detection using OpenCV

The final phase involved integrating the trained model with a real-time computer vision pipeline.

1. Loading the Model

The pkl model and the label encoder were loaded in Python. This enabled the system to classify new landmark-based inputs.

2. Opening the Webcam

OpenCV's VideoCapture(0) function was used to access the laptop or PC camera.

3. Detecting Faces

Using dlib's frontal face detector, faces were identified in each frame of the video stream.

4. Extracting Landmarks for Live Faces

The same 68 landmark predictor used during training was applied to each detected face.

5. Predicting Emotion

- The landmark coordinates were reshaped to match the model's input shape.
- The model generated a prediction.
- The prediction was decoded using the label encoder into an emotion name.

6. Displaying Results

Emotions were displayed on the video frame using OpenCV's putText(). This produced a fully functional real-time emotion detection system capable of classifying the user's facial expressions instantly.

3.7 SIGNIFICANCE OF THE PROJECT

This project demonstrates a practical and efficient method for AI-based emotion analysis without requiring heavy GPU resources. It combines:

- Computer vision (OpenCV)
- Landmark extraction (dlib)
- Machine learning (scikit-learn)
- Real-time inference

Such systems can be used in:

- Student attention monitoring
- Security/surveillance
- Customer emotion analysis
- Healthcare mood tracking
- Human-computer interaction

The project highlights how classical ML techniques can still solve complex real-world problems when applied intelligently.

3.8 SUMMARY

The Facial Emotion Detection System was successfully implemented using a three-phase architecture:

1. **Landmark extraction from Kaggle dataset images**
2. **Model training using scikit-learn and saving model files**
3. **Real-time emotion detection using OpenCV and Python**

The project combines simplicity, efficiency, and practicality, demonstrating the power of Python-based machine learning pipelines.

CHAPTER – 4

SNAPSHOTS OF PROJECT MODULES

Datasets folder structure :

Landmarks_extraction.py :

```python
import os
import cv2
import mediapipe as mp
import numpy as np
import csv
import pandas as pd

DATASET_PATH = "dataset/train"
output_file = "landmarks_train.csv"

# Load already processed file if exists
processed_files = set()
if os.path.exists(output_file):
    try:
        df = pd.read_csv(output_file)
        processed_files = set(df["label"].astype(str) + "_" + df.index.astype(str))
        print(f"✅ Resuming: {len(processed_files)} samples already processed.")
    except:
        print("⚠️ Error reading existing CSV, starting fresh.")

mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(static_image_mode=True, max_num_faces=1)

# If file doesn't exist → create header
if not os.path.exists(output_file):
    with open(output_file, "w", newline="") as file:
        writer = csv.writer(file)
        header = []
        for i in range(468):
            header += [f"x{i}", f"y{i}", f"z{i}"]
        header.append("label")
        writer.writerow(header)

emotions = os.listdir(DATASET_PATH)
```

```python
with open(output_file, "a", newline="") as file:
    writer = csv.writer(file)

    for label in emotions:
        folder = os.path.join(DATASET_PATH, label)

        if not os.path.isdir(folder):
            continue

        print(f"Processing: {label}")

        for img_name in os.listdir(folder):
            unique_id = f"{label}_{img_name}"

            # Skip already processed images
            if any(unique_id in s for s in processed_files):
                continue

            img_path = os.path.join(folder, img_name)
            img = cv2.imread(img_path)
            if img is None:
                continue

            img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
            results = face_mesh.process(img_rgb)

            if results.multi_face_landmarks:
                face = results.multi_face_landmarks[0]
                row = []

                for lm in face.landmark:
                    row.extend([lm.x, lm.y, lm.z])

                row.append(label)
                writer.writerow(row)

```

Landmarks_training.py :

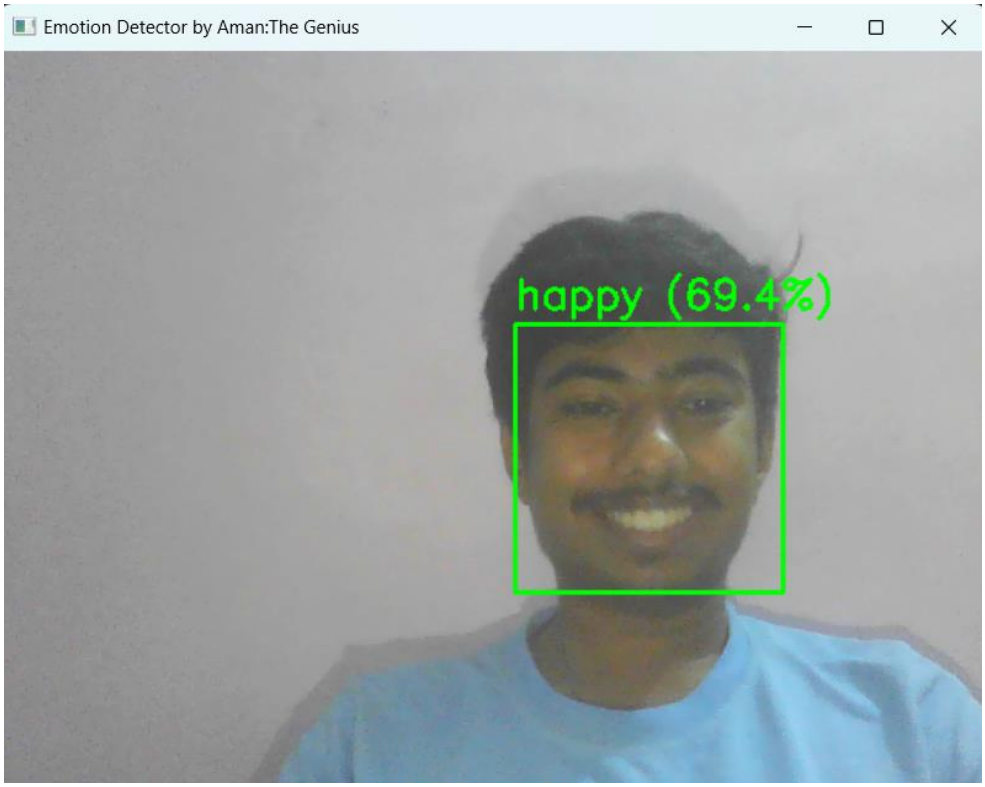
```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.neural_network import MLPClassifier
import joblib

# Load data
data = pd.read_csv("landmarks_train.csv")

X = data.iloc[:, :-1].values
y = data.iloc[:, -1].values

# Encode labels
encoder = LabelEncoder()
y = encoder.fit_transform(y)

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Model
model = MLPClassifier(hidden_layer_sizes=(512, 256), activation='relu', max_iter=300, verbose=True)

print("⏳ Training model...")
model.fit(X_train, y_train)

print("✅ Training accuracy:", model.score(X_train, y_train))
print("✅ Testing accuracy:", model.score(X_test, y_test))

# Save model + label encoder
joblib.dump(model, "landmark_emotion_model.pkl")
joblib.dump(encoder, "label_encoder.pkl")

print("🎉 Model saved successfully!")
```

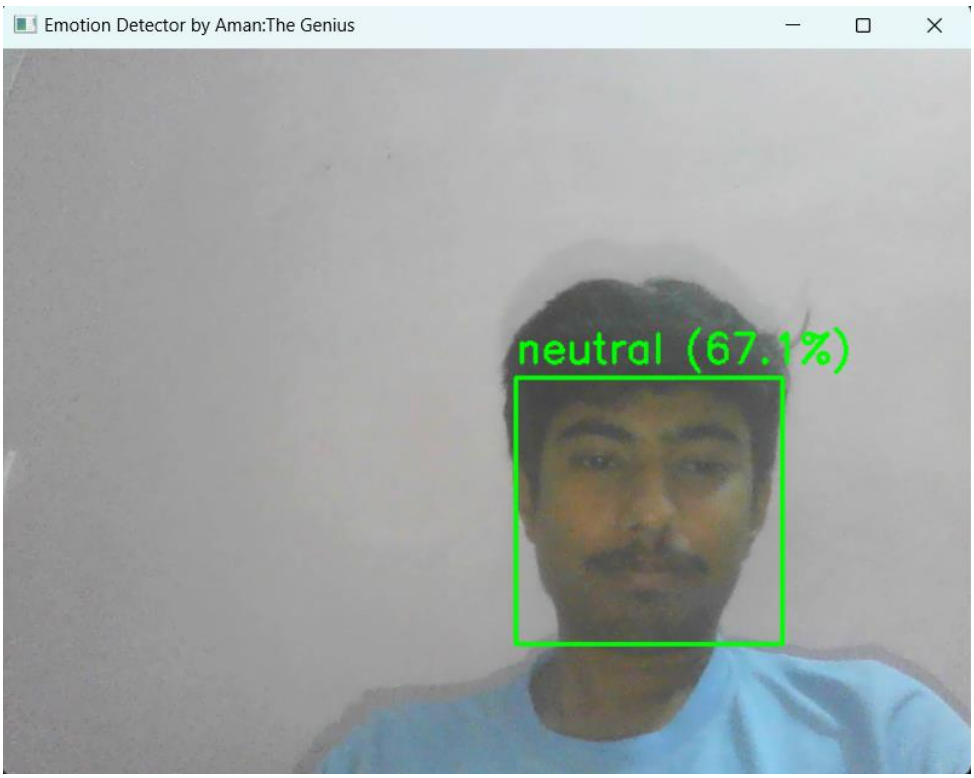
Emotion_detection.py :

```python
import cv2
import mediapipe as mp
import numpy as np
from collections import deque, Counter
import joblib

# Load trained model + label encoder
model = joblib.load("landmark_emotion_model.pkl")
label_encoder = joblib.load("label_encoder.pkl")

# Smoothing window
SMOOTH_WINDOW = 7
recent_predictions = deque(maxlen=SMOOTH_WINDOW)

# Mediapipe setup
mp_face_mesh = mp.solutions.face_mesh
mp_face_detection = mp.solutions.face_detection

face_mesh = mp_face_mesh.FaceMesh(
    static_image_mode=False,
    max_num_faces=1,
    refine_landmarks=False,
    min_detection_confidence=0.6,
    min_tracking_confidence=0.6
)

face_detector = mp_face_detection.FaceDetection(
    model_selection=1,
    min_detection_confidence=0.6
)

def smooth_label(queue):
    if not queue:
        return None
    count = Counter(queue)
    return count.most_common(1)[0][0]

cap = cv2.VideoCapture(0)
print("Running... Press 'q' to quit.")
```

```python
while True:
    ret, frame = cap.read()
    if not ret:
        break

    h, w, _ = frame.shape
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    detection = face_detector.process(rgb)

    if detection.detections:
        det = detection.detections[0]
        box = det.location_data.relative_bounding_box

        x1 = int(box.xmin * w)
        y1 = int(box.ymin * h)
        x2 = x1 + int(box.width * w)
        y2 = y1 + int(box.height * h)

        x1, y1 = max(0, x1), max(0, y1)
        x2, y2 = min(w, x2), min(h, y2)

        face_region = rgb[y1:y2, x1:x2]
        mesh_result = face_mesh.process(face_region)

        if mesh_result.multi_face_landmarks:
            lm = mesh_result.multi_face_landmarks[0].landmark

            lm = lm[:468]

            landmarks = []
            for p in lm:
                landmarks.extend([p.x, p.y, p.z])

            input_data = np.array(landmarks).reshape(1, -1)

            probs = model.predict_proba(input_data)[0]
            emotion = label_encoder.inverse_transform([np.argmax(probs)])[0]
            confidence = np.max(probs) * 100

            recent_predictions.append(emotion)
            display_emotion = smooth_label(recent_predictions)

            cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
            cv2.putText(frame, f"{display_emotion} ({confidence:.1f}%)",
                        (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX,
                        0.9, (0, 255, 0), 2)

    else:
        cv2.putText(frame, "Face Not Detected",
                    (20, 40), cv2.FONT_HERSHEY_SIMPLEX,
                    0.9, (0, 0, 255), 2)

```

Output :


Emotion Detector by Aman:The Genius
happy (69.4%)


Emotion Detector by Aman:The Genius
neutral (67.1%)

CHAPTER – 5
CONCLUSION

The project titled **"Facial Emotion Detection System using Machine Learning and Python"** was successfully developed and implemented. The system accurately identifies human emotions in real time by combining facial landmark extraction, machine learning using the **MLP Classifier**, and computer vision through **OpenCV**. By converting facial expressions into 68 landmark coordinates and training a neural-network-based model, the project achieved reliable and efficient performance without requiring heavy computational resources.

Throughout the development process, key concepts such as data preprocessing, landmark-based feature engineering, model training, evaluation, and real-time inference were explored. The project provided valuable hands-on experience in Python-based AI workflows and strengthened my understanding of practical machine learning pipelines.

Overall, this work demonstrates how lightweight ML models can be effectively used for real-time emotion recognition. It sets a solid foundation for future improvements, such as integrating deep learning, analyzing multiple faces simultaneously, or deploying the system on cloud and mobile platforms. The knowledge gained through this project will be highly beneficial for future studies and professional growth in AI and Machine Learning.

CHAPTER – 6
REFERENCES

1. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research*, 12, 2825–2830, 2011.
2. O'Reilly Media. "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow."
   Aurélien Géron, 2019.
3. Kaggle. "Facial Emotion Recognition Dataset."
   Available at: https://www.kaggle.com (Accessed 2025).
4. Bradski, G. "The OpenCV Library." *Dr. Dobb's Journal of Software Tools*, 2000.